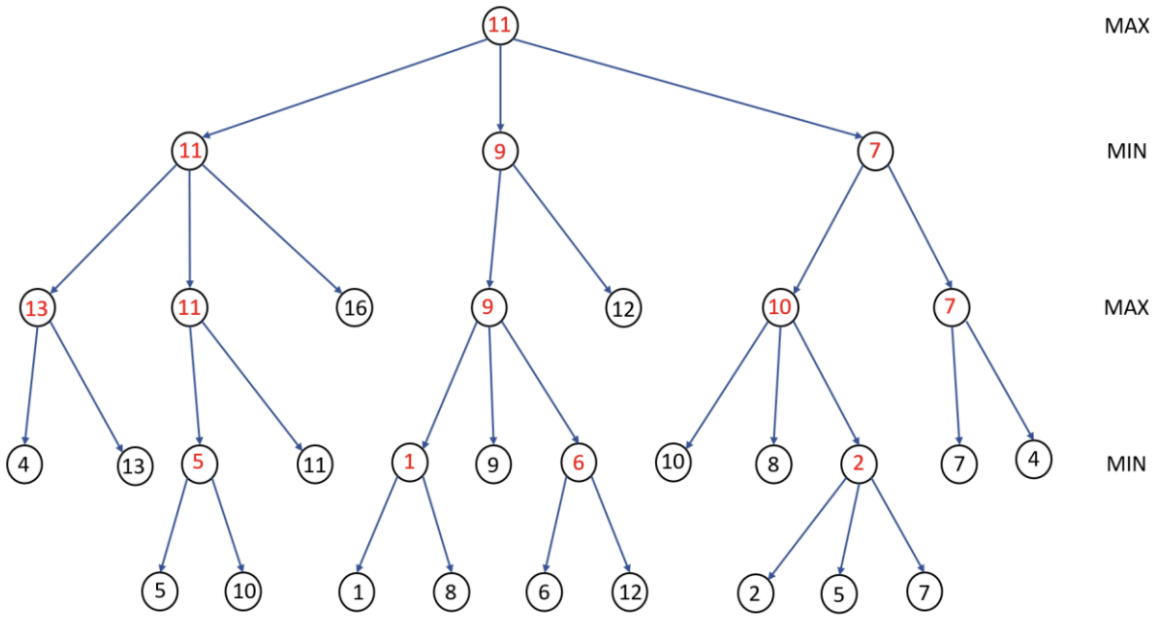
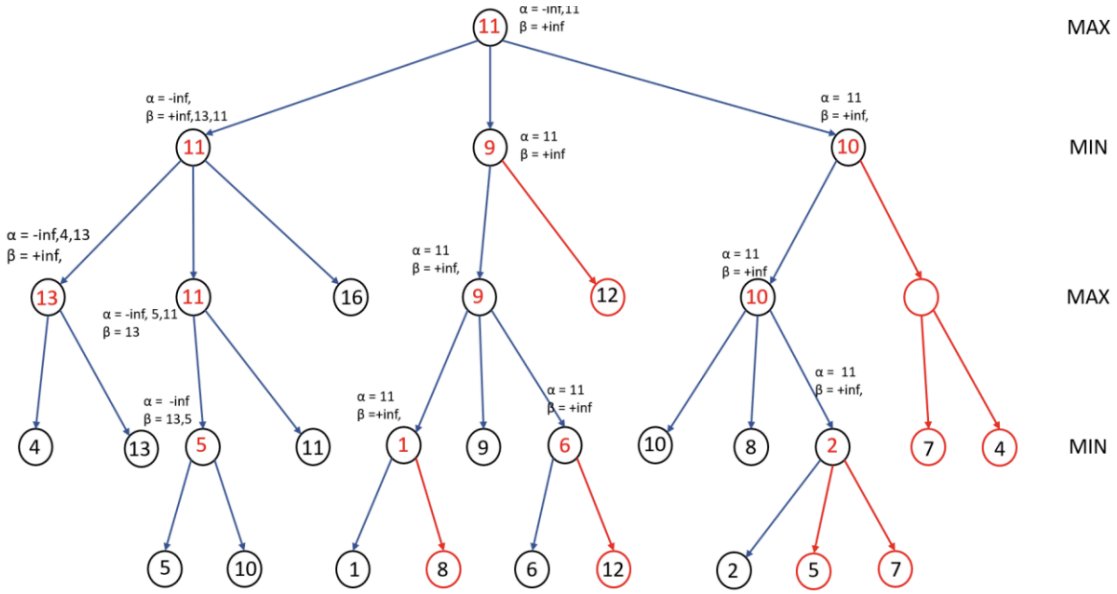
AI 500 - Set 1 - Solutions

Q1a)



Q1b)

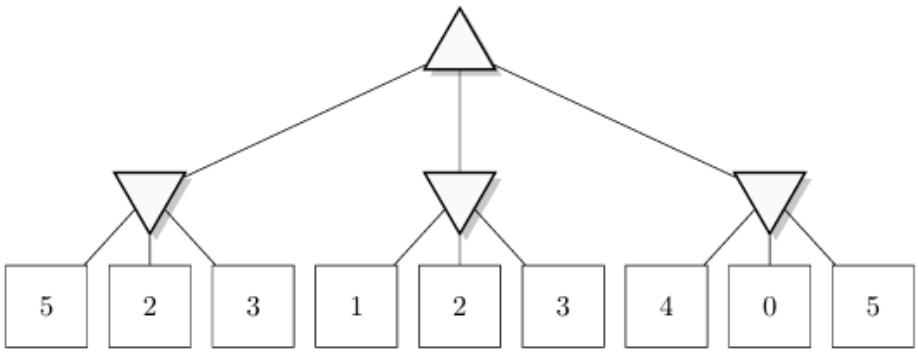


Q1c)

No

Q2

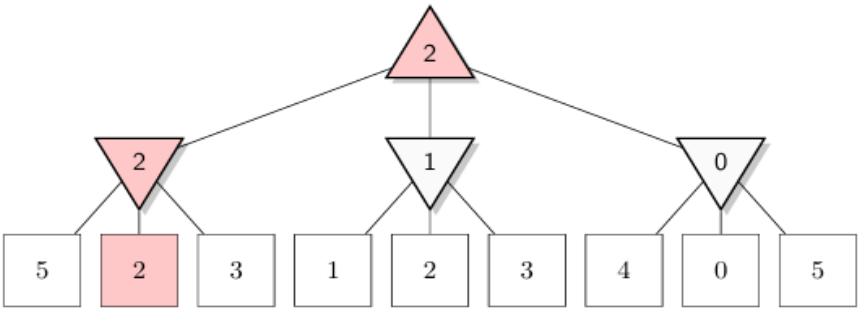
Consider the following tree representing a zero-sum game, where triangles pointing up are max nodes, triangles pointing down are min nodes, and squares are terminal nodes with the corresponding value of utility function for max player.



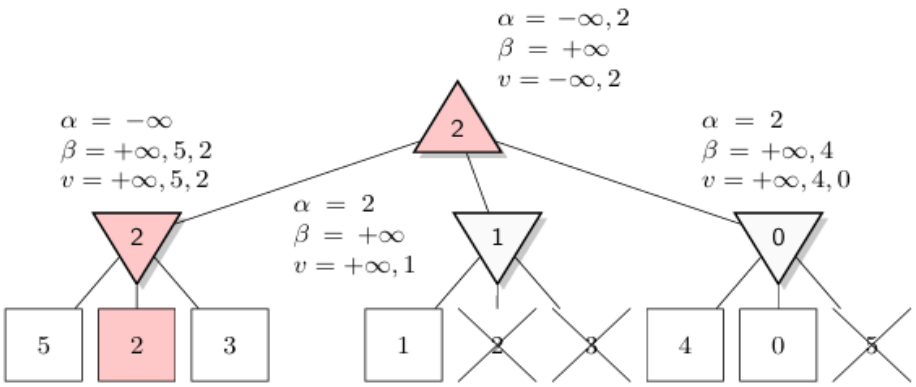
1. Apply the minimax algorithm for finding the best action for the max player at the root.
2. Apply the minimax algorithm with alpha-beta pruning for finding the best action for the max player at the root.

Ans:

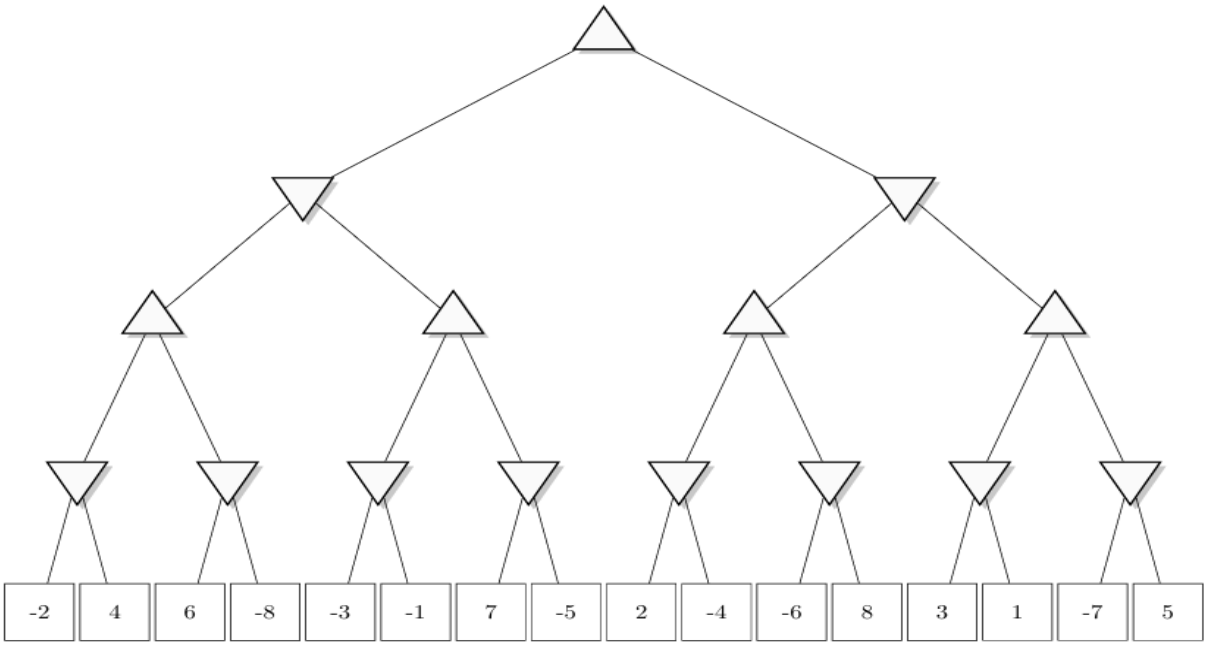
Minimax solution



Minimax solution with alpha-beta pruning



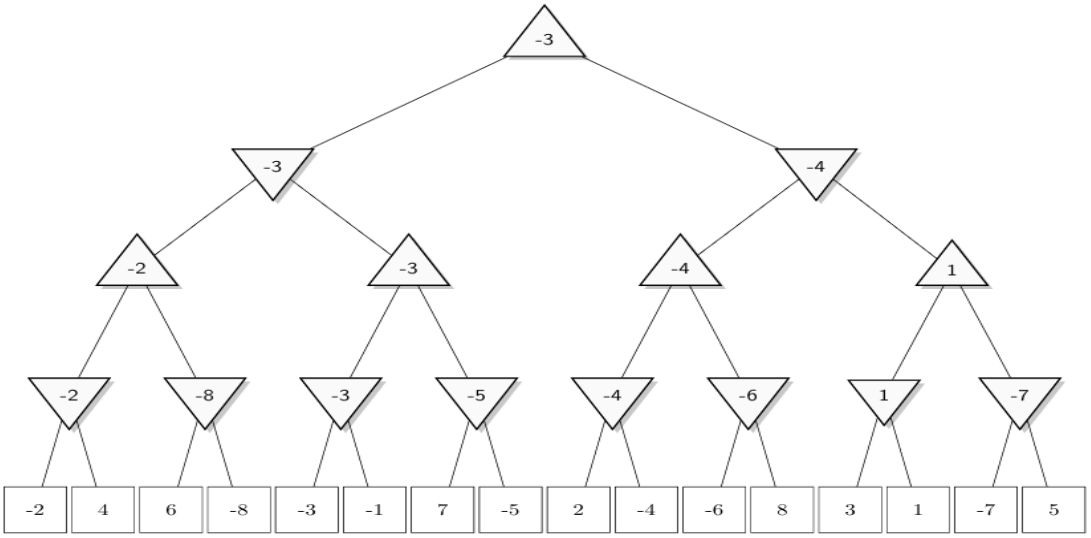
Q3. Consider the following tree representing a zero-sum game, where triangles pointing up are max nodes, triangles pointing down are min nodes, and squares are terminal nodes with the corresponding value of utility function for max player:



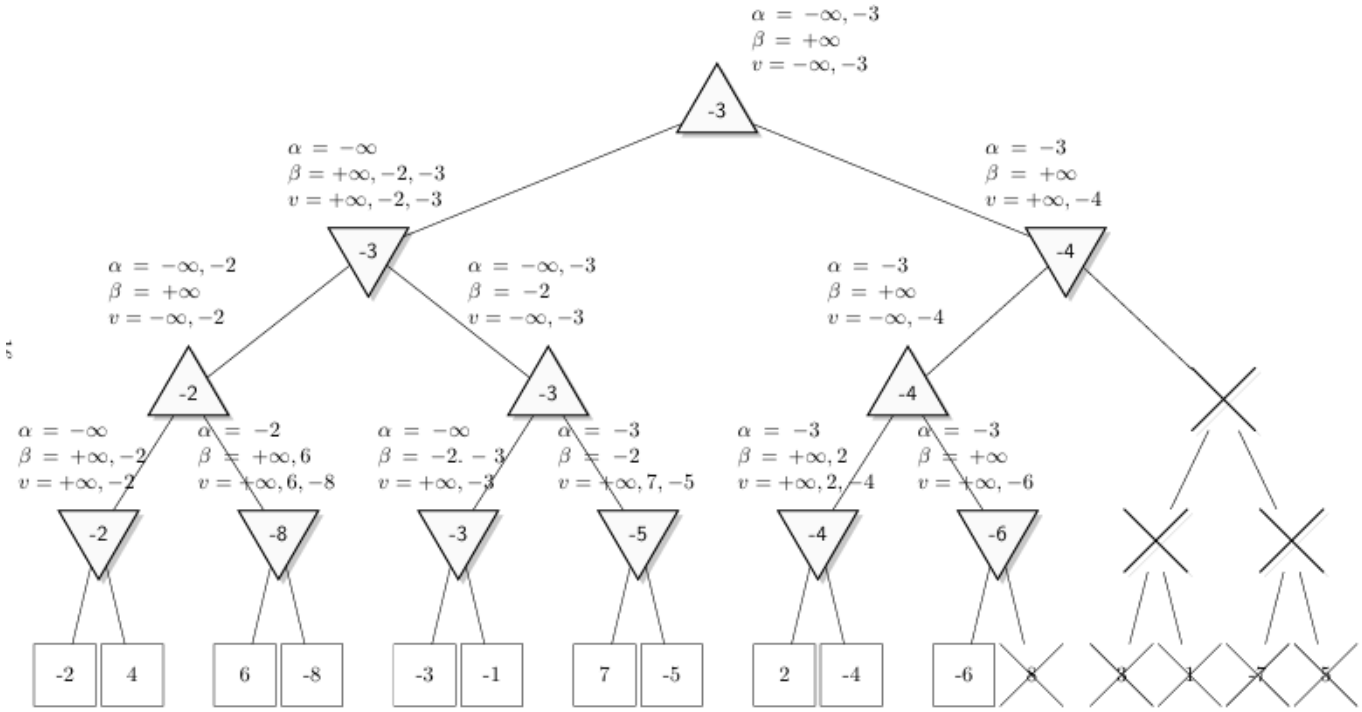
1. Apply the minimax algorithm for finding the best action for the max player at the root.
2. Apply the minimax algorithm with alpha-beta pruning for finding the best action for the max player at the root.

Ans

1.



2.



Question 4: Minimax

Consider a MIN node whose children are terminal nodes. If MIN plays suboptimally, then the value of the node is greater than or equal to the value it would have if MIN played optimally. Hence, the value of the MAX node that is the MIN node's parent can only be increased. This argument can be extended by a simple induction all the way to the root. If the suboptimal play by MIN is predictable, then one can do better than a minimax strategy. For example, if MIN always falls for a certain kind of trap and loses, then setting the trap guarantees a win even if there is actually a devastating response for MIN.

Question 5: RBFS

1. *What is the main difference between RBFS and A in terms of memory usage?**

A* keeps all generated nodes in memory (open list + closed list), which can grow exponentially with the search depth.
RBFS (Recursive Best-First Search), on the other hand, only keeps track of a limited number of nodes along the current path plus some information (the best alternative f-values) for backtracking. Therefore, RBFS uses linear space in the depth of the search tree, unlike A* which can run out of memory on large problems.

2. *In RBFS, why do we store the "best alternative f-value" for each node?*

RBFS may need to backtrack when the current path becomes worse than an alternative. To avoid re-expanding the same subtree from scratch, RBFS stores the best alternative f-value for each node. This acts as a kind of "bookmark" so that when the algorithm returns to that node, it remembers the cost of the next-best option without having to recompute it.

3. *Does RBFS always guarantee finding an optimal solution? Justify your answer.*

Yes, RBFS is optimal *provided that the heuristic used is admissible (never overestimates the true cost)*. Like A*, RBFS always expands nodes in order of increasing f-value and keeps track of alternatives when backtracking. It will eventually explore the optimal path as long as the heuristic is admissible and consistent.

Question 6: Beam Search (NLP)

Solution part i):

*(a) → **A (0.5)**, **B (0.3)**.*

(b)

- *From A (0.5):*
  - *AD = 0.5 × 0.4 = **0.20***
  - *A <END> = 0.5 × 0.5 = **0.25***
- *From B (0.3):*
  - *BC = 0.3 × 0.6 = **0.18***
  - *B <END> = 0.3 × 0.2 = **0.06***

Rank all → A <END> (0.25), AD (0.20), BC (0.18), B <END> (0.06).
*Top-2 kept: **A <END> (0.25)**, **AD (0.20)**.*

(c) Completed sequences among these: A <END> (score 0.25).
*So the best completed sequence is **A**.*

Solution part ii):

(a)→ ***X (0.4), Y (0.35), Z (0.25)****.*

(b)

- *From X (0.4):*
  - *XY = 0.4 × 0.5 =* ***0.20***
  - *X <END> = 0.4 × 0.3 =* ***0.12***
- *From Y (0.35):*
  - *YZ = 0.35 × 0.4 =* ***0.14***
  - *Y <END> = 0.35 × 0.4 =* ***0.14***
- *From Z (0.25):*
  - *ZX = 0.25 × 0.6 =* ***0.15***
  - *Z <END> = 0.25 × 0.2 =* ***0.05***

Rank all: XY (0.20), ZX (0.15), YZ (0.14), Y <END> (0.14), X <END> (0.12), Z <END> (0.05).
Top-3 beams: ***XY (0.20), ZX (0.15), YZ (0.14)****.*

(c) Step 3 expansions:

- *XY <END> = 0.20 × 0.7 =* ***0.14***
- *ZX <END> = 0.15 × 0.5 =* ***0.075***
- *YZ <END> = 0.14 × 0.6 =* ***0.084***

Final ranking: XY <END> (0.14), YZ <END> (0.084), ZX <END> (0.075).
Best sequence = ***XY****.*

Question 7: RBFS

Solution: https://youtu.be/ehUkLVZRHkk?si=UYeMeM8mEaqBguPC

Question 8: Beam Search

Step 1 (token1):

- `I = 0.55`
- `You = 0.45`
  → Keep: `I (0.55)`, `You (0.45)` (only 2 available)

Step 2 (token2):

- From `I (0.55)`:
  - `I love = 0.33`
  - `I hate = 0.22`
- From `You (0.45)`:
  - `You love = 0.09`
  - `You hate = 0.36`

Keep top 3:

- `You hate (0.36)`
- `I love (0.33)`
- `I hate (0.22)`

Step 3 (token3):

- From `You hate (0.36)`:
  - `You hate cats = 0.036`

    - `You hate dogs = 0.324`

- From `I love (0.33)`:
    - `I love cats = 0.165`
    - `I love dogs = 0.165`
- From `I hate (0.22)`:
    - `I hate cats = 0.022`
    - `I hate dogs = 0.198`

Top 3 final sequences:

1. `You hate dogs (0.324)`
2. `I hate dogs (0.198)`
3. `I love cats (0.165)` (tie with dogs, assume cats)

Final Beam=3 Result:

- Best sequence: **“You hate dogs” (0.324)**
- Next: **“I hate dogs” (0.198)**
- Next: **“I love cats” (0.165)**

Question 9: Beam Search

Step 1 (token1):

- **`She = 0.6`**

- `He = 0.4`

Keep top 2: `She (0.6)`, `He (0.4)`

Step 2 (token2):

- **From** `She (0.6)`**:**
    - `She likes = 0.6 × 0.7 = 0.42`
    - `She hates = 0.6 × 0.3 = 0.18`
- **From** `He (0.4)`**:**
    - `He likes = 0.4 × 0.5 = 0.20`
    - `He hates = 0.4 × 0.5 = 0.20`

Keep top 2:

- `She likes (0.42)`
- `He likes (0.20)` **(tie with** `He hates`**, assume** `He likes`**)**

Step 3 (token3):

- **From** `She likes (0.42)`**:**
    - `She likes pizza = 0.42 × 0.4 = 0.168`
    - `She likes sushi = 0.42 × 0.6 = 0.252`
- **From** `He likes (0.20)`**:**
    - `He likes pizza = 0.20 × 0.4 = 0.080`

    - `He likes sushi = 0.20 × 0.6 = 0.120`

Keep top 2:

- `She likes sushi (0.252)`
- `She likes pizza (0.168)`

Final Beam = 2 Result:

1. **Best sequence: “She likes sushi” with probability 0.252**
2. **Second sequence: “She likes pizza” with probability 0.168**